

InstaBuy – Distributed e-Commerce Platform

Internship Project Report

InstaBuy

Distributed Microservices-Based e-Commerce Platform

Submitted by:

Katau Rupesh Reddy

Technology Domain:

Full Stack Development & Microservices Architecture

Frontend Technologies:

React.js, Bootstrap, Axios

Backend Technologies:

Spring Boot, Spring Cloud Gateway, Netflix Eureka

Messaging & Distributed Systems:

Apache ActiveMQ, Saga Pattern

Database Technologies:

MySQL, MongoDB

Payment Integration:

Stripe API

Note:

- Service-Specific Documentation is available at the link provided below:

[InstaBuy e-Commerce Website](#)

- GitHub Link:

[InstaBuy GitHub Link](#)

ABSTRACT

InstaBuy is a scalable and distributed e-commerce platform developed during the internship period at Mphasis. The project was designed to demonstrate the implementation of modern microservices architecture, asynchronous communication, secure payment processing, and distributed transaction management in a real-world online shopping environment.

The platform separates major business functionalities such as authentication, product management, inventory handling, ordering, and payment processing into independently deployable microservices. This modular architecture improves system scalability, fault tolerance, and maintainability while reducing dependency between services.

The frontend application was developed using React.js to provide a responsive and user-friendly shopping experience. Backend services were implemented using Spring Boot and Spring Cloud technologies. Netflix Eureka was used for dynamic service discovery, while Apache ActiveMQ enabled asynchronous event-driven communication between services.

To ensure consistency across distributed services, the system implements the Saga choreography pattern. This approach allows services to communicate through events while maintaining eventual consistency without relying on distributed locks or blocking operations.

The project also integrates Stripe payment services for secure online transactions and uses JWT-based authentication with Role-Based Access Control (RBAC) for system security.

Overall, InstaBuy demonstrates how modern cloud-native technologies and distributed system principles can be effectively applied to develop a scalable, resilient, and production-ready e-commerce platform.

TABLE OF CONTENTS

1. Introduction
2. Project Objectives
3. Problem Statement
4. Existing System
5. Proposed System
6. System Architecture
7. Technology Stack
8. Frontend Implementation
9. Backend Implementation
10. Database Design
11. Service Discovery using Eureka
12. API Gateway
13. Event-Driven Communication using ActiveMQ
14. Saga Pattern Implementation
15. REST APIs
16. Role-Based Access Control (RBAC)
17. Payment Integration
18. Supporting Background Services
19. Security Mechanisms
20. Advantages of the System
21. Challenges Faced During Development
22. Future Enhancements
23. Conclusion

1. INTRODUCTION

The rapid advancement of internet technologies and digital commerce has transformed the way businesses operate and customers purchase products. Modern consumers expect online shopping platforms to provide fast performance, secure transactions, uninterrupted availability, and seamless user experience across multiple devices. As the number of users and transactions increases, traditional monolithic e-commerce systems often struggle to maintain scalability, flexibility, and operational efficiency.

To overcome these limitations, modern software industries are increasingly adopting microservices architecture and event-driven communication models. These architectures allow large applications to be divided into smaller independent services that can be developed, deployed, and scaled separately. Such systems improve fault tolerance, maintainability, and scalability while reducing dependency between components.

During the internship at Mphasis, the project “InstaBuy” was developed as a distributed e-commerce platform based on microservices architecture. The main objective of the project was to understand and implement enterprise-level distributed system concepts using modern backend and frontend technologies.

InstaBuy is designed as a scalable and resilient online shopping platform where core functionalities such as authentication, product management, inventory handling, order processing, and payment management operate as independent microservices. These services communicate asynchronously using Apache ActiveMQ, enabling loose coupling and improving overall system performance.

The platform also integrates technologies such as Spring Boot, React.js, Netflix Eureka, MongoDB, MySQL, Stripe API, JWT Authentication, and Spring Cloud Gateway to simulate a real-world cloud-native enterprise application.

The project demonstrates how distributed systems can effectively handle complex business workflows such as order processing, payment confirmation, inventory reservation, and transaction recovery while maintaining consistency across multiple services.

In addition to core e-commerce functionalities, InstaBuy also incorporates advanced enterprise features such as:

- Distributed transaction management using the Saga Pattern
- Role-Based Access Control (RBAC)
- Dynamic service discovery
- Event-driven asynchronous communication
- Payment gateway integration

- Dead Letter Queue (DLQ) handling
- Retry mechanisms and failure recovery
- Vendor sales analytics
- Automated invoice generation

The implementation of these technologies provided practical exposure to real-world software engineering practices followed in enterprise environments.

Overall, InstaBuy represents a modern distributed e-commerce solution that combines scalability, reliability, security, and maintainability using cloud-native architectural principles.

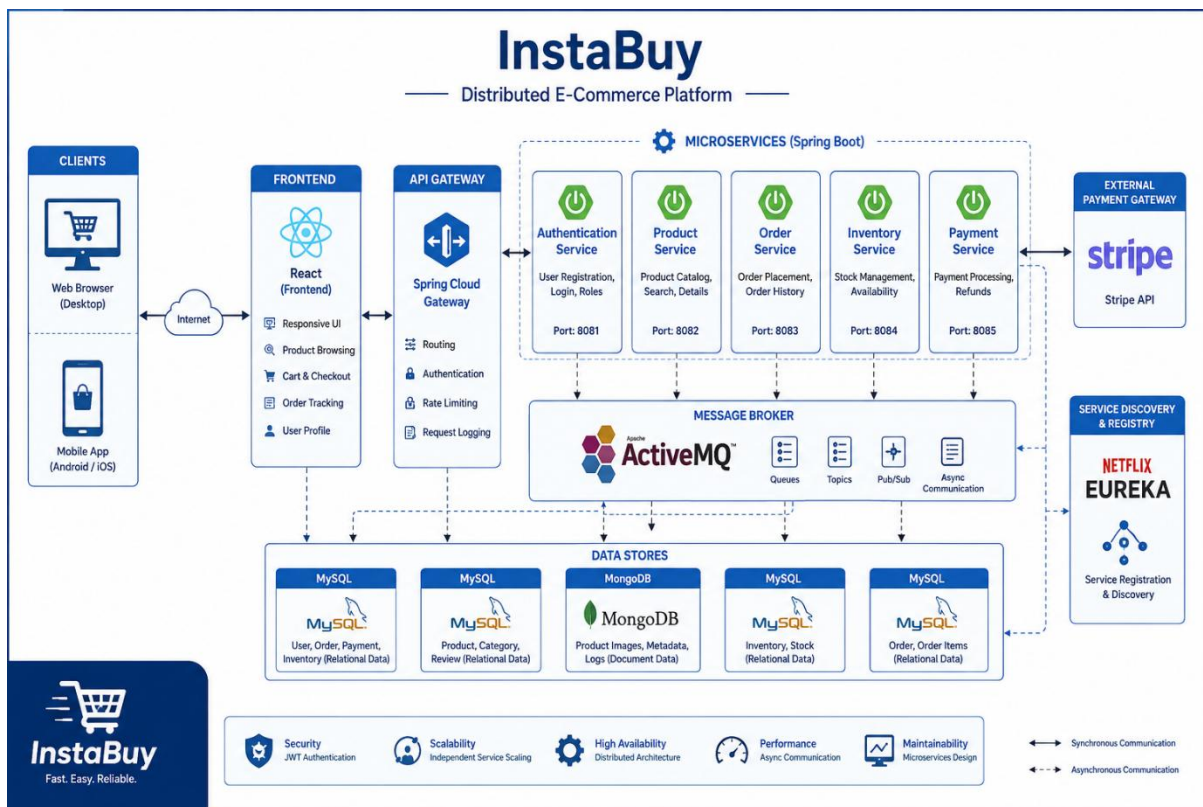


Figure 1: Overview of InstaBuy Distributed e-Commerce Platform

2. PROJECT OBJECTIVES

The primary objective of the InstaBuy project was to design and develop a scalable distributed e-commerce platform capable of handling modern business requirements using microservices architecture and asynchronous communication mechanisms.

The project was developed to gain practical understanding of enterprise-level software architecture and distributed system design during the internship period at Mphasis.

Unlike traditional monolithic systems, the project focused on implementing independently deployable services that communicate efficiently while maintaining system consistency and reliability.

The major objectives of the project are explained below.

2.1 Development of Microservices Architecture

One of the core objectives of the project was to implement a distributed microservices-based architecture where each business functionality operates as an independent service.

The project separates major modules such as:

- Authentication
- Product Management
- Inventory Handling
- Order Processing
- Payment Management

into standalone services.

This separation improves:

- Scalability
- Independent deployment
- Fault isolation
- Maintainability
- Flexibility for future upgrades

The microservices approach also enables different services to scale independently depending on system load.

2.2 Implementation of Event-Driven Communication

Another important objective was to establish asynchronous communication between services using Apache ActiveMQ.

Instead of relying completely on synchronous REST communication, the project uses event-driven messaging for critical business workflows such as:

- Order processing
- Inventory reservation
- Payment confirmation
- Failure recovery

This approach reduces tight coupling between services and improves overall system resilience.

The event-driven model also enables better handling of failures and retry mechanisms in distributed environments.

2.3 Distributed Transaction Management Using Saga Pattern

Maintaining consistency across multiple services is one of the biggest challenges in distributed systems.

To address this issue, InstaBuy implements the Saga choreography pattern for managing distributed transactions.

The objective of implementing Saga was to:

- Maintain eventual consistency
- Avoid distributed locking
- Prevent blocking operations
- Enable rollback and compensation handling

This helped the platform manage complex workflows such as payment processing and inventory updates efficiently.

2.4 Secure Authentication and Authorization

Security was another major focus area of the project.

The project aimed to implement:

- JWT-based authentication
- BCrypt password encryption
- Role-Based Access Control (RBAC)

These mechanisms ensure that only authorized users can access protected functionalities within the system.

Different access permissions were implemented for:

- Customers
- Vendors
- Administrators

This improved overall platform security and access management.

2.5 Payment Gateway Integration

The project also focused on integrating secure online payment processing using Stripe API.

The payment system was designed to support:

- Secure transaction processing
- Payment verification
- Webhook handling
- Transaction status updates

This provided practical understanding of integrating third-party payment services within distributed applications.

2.6 Improving System Scalability and Reliability

A major objective of InstaBuy was to build a platform capable of handling increasing user traffic and service failures without affecting overall system performance.

To achieve this, the project implemented:

- Dynamic service discovery
- Asynchronous messaging
- Retry mechanisms
- Dead Letter Queues (DLQ)

- Independent service scaling

These features improved the availability and fault tolerance of the platform.



Figure 2: Major Objectives and Components of InstaBuy

3. PROBLEM STATEMENT

Traditional e-commerce applications are generally developed using monolithic architecture where all functionalities are tightly integrated into a single application. While this approach may work for small-scale systems, it becomes increasingly difficult to manage as the application grows in size and complexity.

Modern e-commerce platforms are expected to support:

- Large numbers of concurrent users
- Continuous availability
- Secure payment processing
- Fast response times
- Independent feature updates

- Fault tolerance and recovery

Monolithic systems often fail to meet these requirements efficiently.

One of the major problems with traditional systems is tight coupling between modules. Since all functionalities are interconnected, a failure in one component can affect the entire application. Deployment also becomes difficult because even small updates require redeploying the complete system.

Scalability is another major issue. In monolithic systems, the entire application must scale together, even if only one module experiences heavy traffic. This increases infrastructure cost and reduces efficiency.

Additionally, synchronous communication between components creates blocking operations that reduce responsiveness during high system load.

The challenge addressed by InstaBuy was to design a distributed e-commerce platform capable of:

- Independent service scalability
- High availability
- Distributed transaction consistency
- Fault isolation
- Secure payment handling
- Event-driven asynchronous communication

The system also needed to handle failure scenarios such as:

- Payment failures
- Inventory shortages
- Service downtime
- Message delivery retries
- Order rollback handling

To solve these problems, InstaBuy adopted modern distributed system principles using microservices architecture, ActiveMQ messaging, Saga-based transaction management, and cloud-native service discovery mechanisms.

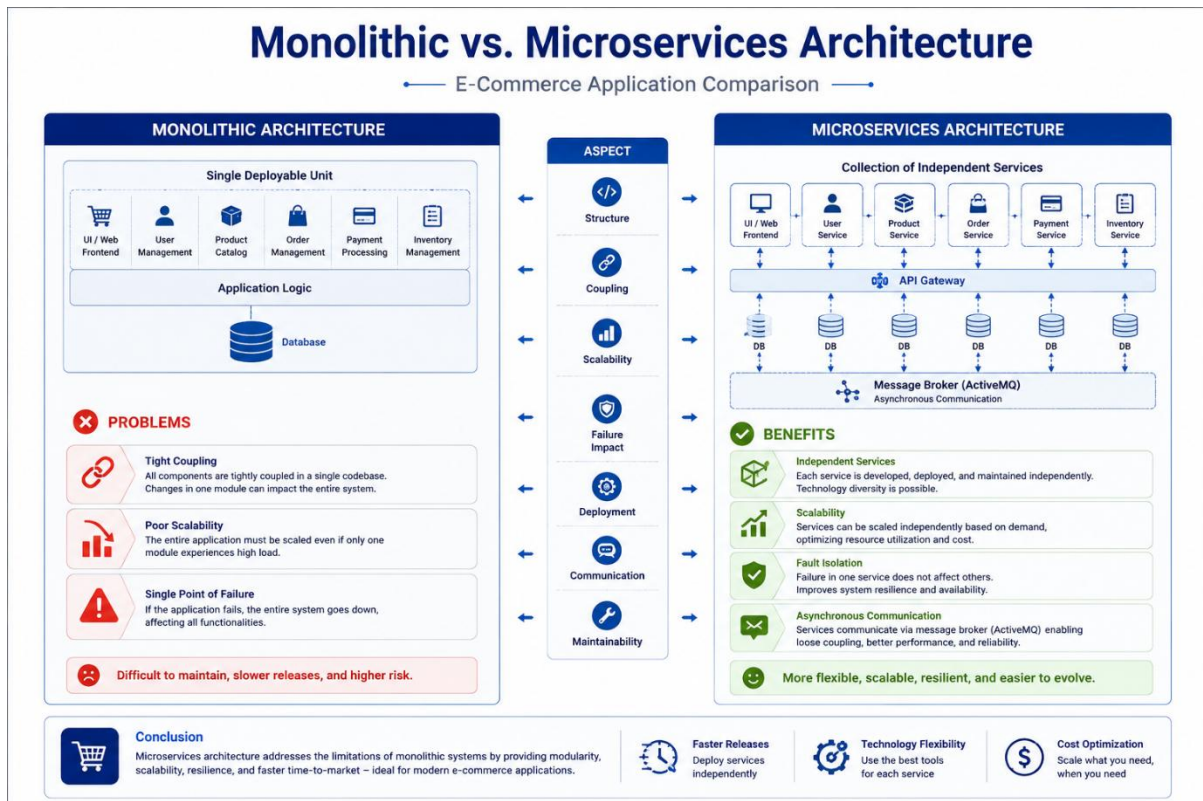


Figure 3: Comparison Between Monolithic and Microservices Architecture

4. EXISTING SYSTEM

Traditional e-commerce platforms are commonly developed using monolithic architecture, where all functionalities such as authentication, product management, order processing, inventory handling, and payment operations are tightly integrated into a single application. Although this architecture is simpler during the initial stages of development, it becomes increasingly difficult to maintain and scale as the application grows.

In a monolithic system, all modules share the same codebase, database, and deployment environment. As a result, even a small modification in one module requires rebuilding and redeploying the entire application. This increases deployment complexity and slows down development cycles.

Another major limitation of monolithic systems is poor fault isolation. Since all functionalities are interconnected, failure in one component can affect the stability of the complete application. For example, if the payment module experiences failure, it may impact product browsing or order management functionalities as well.

Scalability also becomes inefficient in monolithic architectures. During high traffic conditions, the entire application must be scaled even if only one specific module experiences heavy load. This increases infrastructure cost and reduces resource optimization.

Most traditional systems also rely heavily on synchronous communication between modules. This creates blocking operations where one service must wait for another service to respond before continuing execution. Under heavy load conditions, this significantly reduces application performance and responsiveness.

The major limitations identified in traditional e-commerce systems are:

- Tight coupling between modules
- Difficult maintenance and deployment
- Poor scalability
- Single point of failure
- Reduced fault tolerance
- Limited flexibility for technology upgrades
- Complex debugging and monitoring
- High infrastructure cost during scaling

These challenges highlighted the need for a more scalable and resilient architecture capable of supporting modern enterprise-level e-commerce operations.

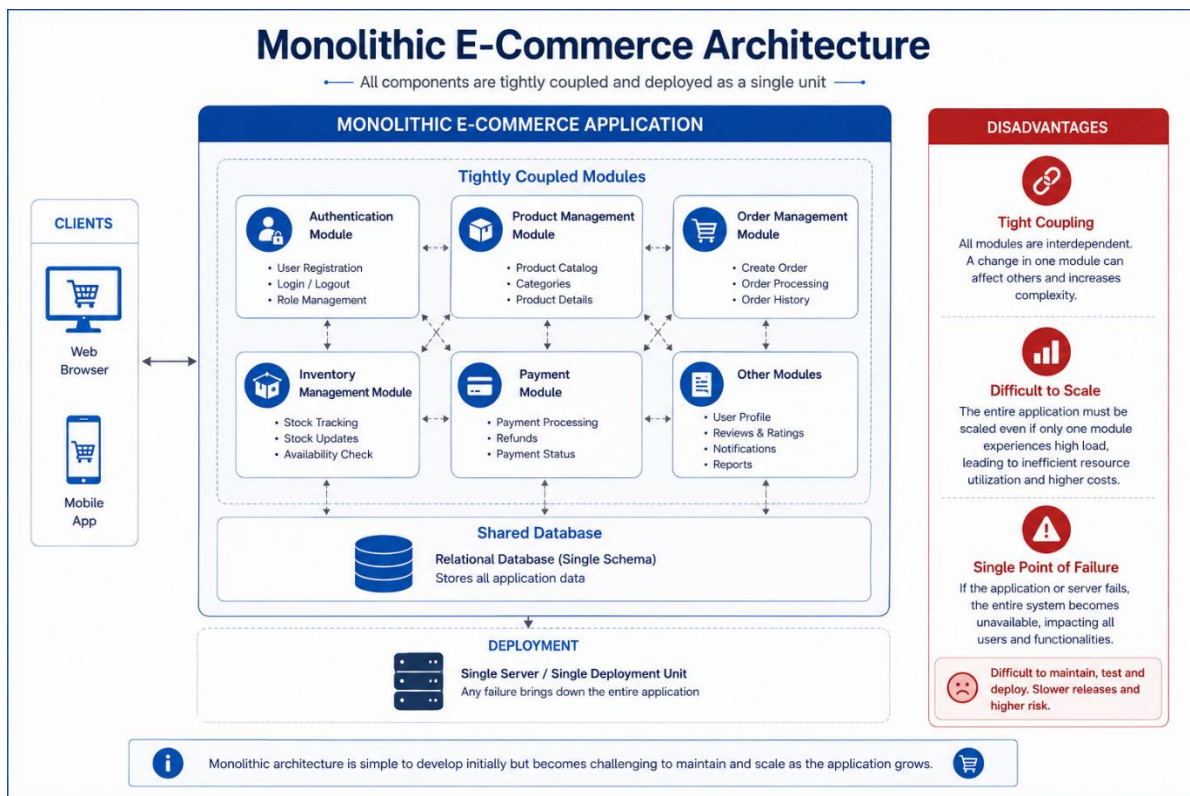


Figure 4: Traditional Monolithic e-Commerce Architecture

5. PROPOSED SYSTEM

To overcome the limitations of traditional monolithic systems, InstaBuy adopts a distributed microservices-based architecture combined with asynchronous event-driven communication mechanisms.

In the proposed system, every major business functionality operates as an independent microservice. Each service has its own responsibilities, database operations, and deployment lifecycle. This separation improves maintainability, scalability, and fault isolation while reducing dependency between components.

The proposed architecture mainly consists of:

- React.js Frontend Application
- Spring Cloud API Gateway
- Authentication Service
- Product Service
- Inventory Service
- Order Service
- Payment Service
- Apache ActiveMQ Messaging Broker
- Netflix Eureka Service Registry
- MySQL and MongoDB Databases

Each microservice communicates either synchronously using REST APIs or asynchronously using ActiveMQ event messaging depending on the business requirement.

The platform also implements the Saga choreography pattern for distributed transaction management. This ensures consistency between services during complex workflows such as payment processing and inventory reservation.

For example, during order placement:

1. The Order Service creates the order
2. Inventory Service reserves stock
3. Payment Service processes payment
4. Events are exchanged asynchronously

5. Failed transactions trigger compensation mechanisms

This approach avoids distributed locking and improves system resilience.

The proposed system provides several advantages over monolithic architecture, including:

- Independent service deployment
- Better scalability
- Improved fault tolerance
- Faster development cycles
- Asynchronous communication
- Easier maintenance
- Better technology flexibility
- Improved distributed transaction handling

The architecture was designed to simulate real-world enterprise cloud-native systems commonly used in modern software industries.

5.1 Key Features of Proposed System

The proposed InstaBuy platform includes several advanced enterprise-level features designed to improve performance, security, and operational reliability.

Distributed Microservices Architecture

Each business functionality is developed as an independent microservice to improve modularity and scalability.

Event-Driven Communication

Apache ActiveMQ enables asynchronous communication between services using queues and topics.

Distributed Transaction Management

The Saga choreography pattern ensures consistency between distributed services.

Dynamic Service Discovery

Netflix Eureka automatically registers and discovers service instances.

API Gateway

Spring Cloud Gateway manages request routing, security filtering, and load balancing.

Secure Authentication

JWT authentication and BCrypt password hashing improve platform security.

Vendor Inventory Management

Vendors can manage product listings, inventory quantities, and pricing.

Stripe Payment Integration

Secure online transactions are handled using Stripe API.

Retry Mechanisms and DLQ Support

Failed messages are retried automatically and moved to Dead Letter Queues when necessary.

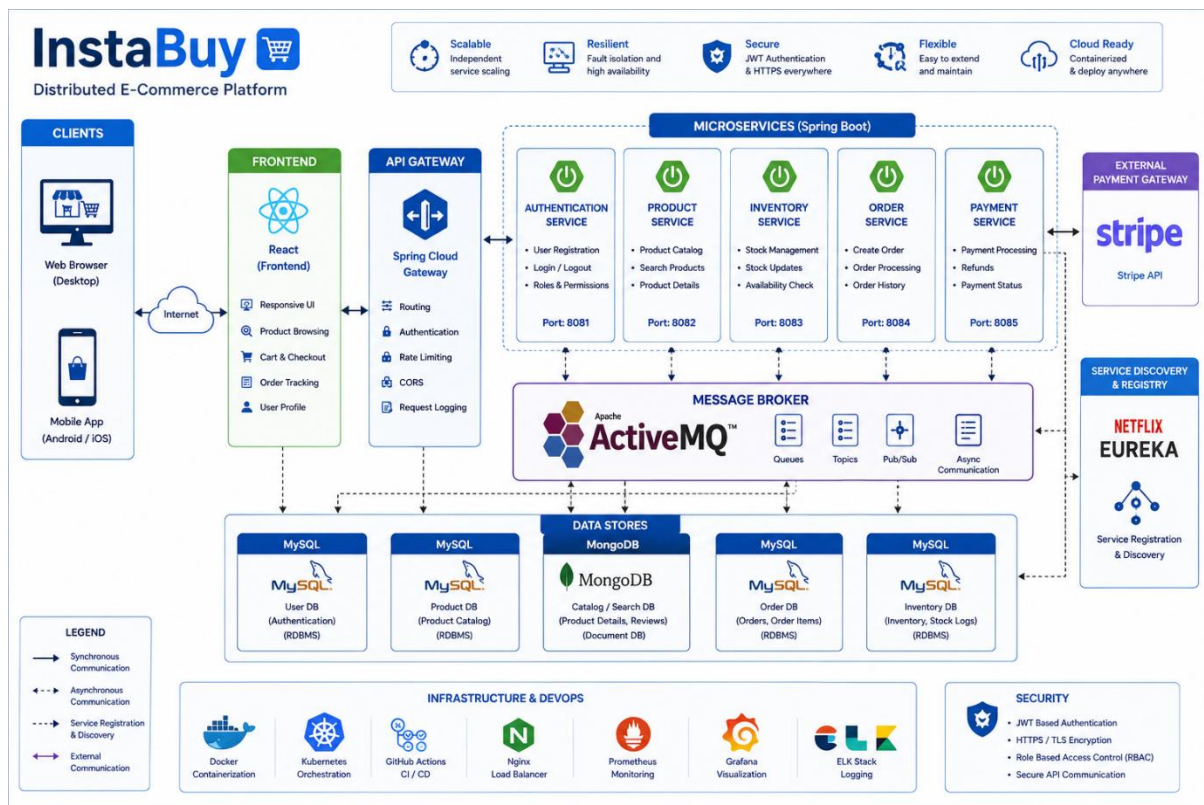


Figure 5: Proposed Distributed Microservices Architecture of InstaBuy

6. SYSTEM ARCHITECTURE

The InstaBuy platform follows a cloud-native distributed system architecture based on microservices principles. The architecture was designed to provide high availability, scalability, fault isolation, and maintainability while supporting asynchronous communication between services.

The complete architecture is divided into multiple layers, each responsible for handling specific functionalities within the system.

The major layers include:

- Presentation Layer
- API Gateway Layer
- Microservices Layer
- Messaging Layer
- Database Layer
- Service Discovery Layer

Each layer interacts with other layers in a controlled and secure manner.

6.1 Presentation Layer

The presentation layer represents the frontend application developed using React.js and Bootstrap 5.

This layer provides interfaces for:

- Customers
- Vendors
- Administrators

The frontend communicates with backend services through REST APIs exposed by the API Gateway.

Major frontend functionalities include:

- User authentication
- Product browsing
- Shopping cart management
- Checkout operations
- Order tracking
- Vendor inventory management
- Sales analytics

The frontend was designed to support responsive interaction across desktop and mobile devices.

6.2 API Gateway Layer

Spring Cloud Gateway acts as the centralized entry point for all incoming client requests.

Instead of allowing clients to directly access backend services, all requests are routed through the API Gateway.

The Gateway performs:

- Request routing
- JWT validation
- Authentication filtering
- Load balancing
- Security enforcement

This improves both system security and traffic management.

6.3 Microservices Layer

The core business functionalities are implemented as independent microservices.

Each microservice contains:

- Controllers
- Business logic
- Database operations
- Security configurations

The major microservices include:

Authentication Service

Handles user registration, login, JWT generation, and role management.

Product Service

Manages product catalog operations and product search functionality.

Inventory Service

Handles stock management and inventory reservation.

Order Service

Processes order creation and order tracking.

Payment Service

Handles Stripe payment processing and payment confirmation events.

The separation of services improves modularity and fault isolation.

6.4 Messaging Layer

Apache ActiveMQ acts as the messaging backbone for asynchronous communication.

Services exchange events through:

- Queues
- Topics

The messaging layer is primarily responsible for:

- Distributed transaction coordination
- Retry handling
- Failure recovery
- Event-based communication

This layer plays a critical role in implementing the Saga pattern.

6.5 Database Layer

The system uses both relational and NoSQL databases.

MySQL

Used for transactional data such as:

- Users
- Orders
- Payments
- Inventory

MongoDB

Used for:

- Product catalogs
- Product descriptions
- Dynamic product attributes

Using multiple databases improved flexibility and performance.

6.6 Service Discovery Layer

Netflix Eureka acts as the service registry within the distributed environment.

Services automatically:

- Register themselves
- Send heartbeat signals
- Discover healthy instances

This enables dynamic scaling and improves fault tolerance.

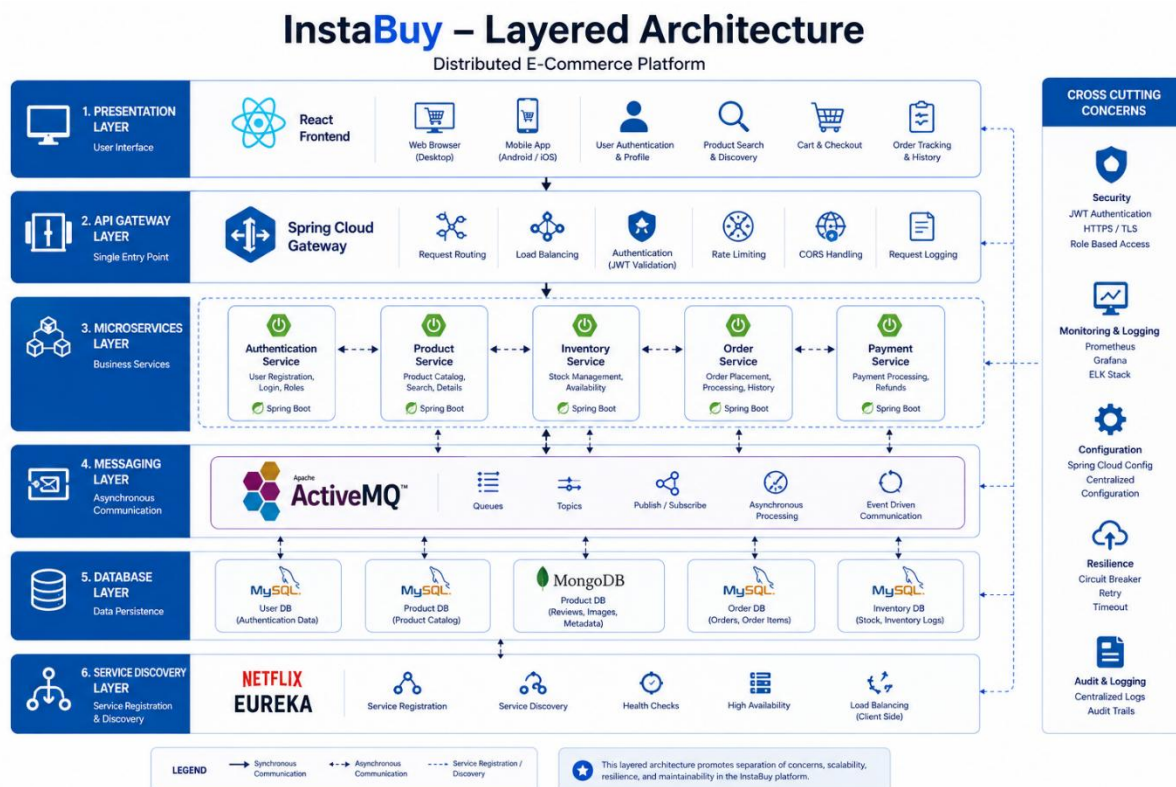


Figure 6: Layered System Architecture of InstaBuy

7. TECHNOLOGY STACK

The InstaBuy platform was developed using a combination of modern frontend, backend, database, messaging, and security technologies. The selection of technologies was based on scalability, performance, flexibility, and enterprise-level industry adoption.

The technology stack was carefully designed to support distributed microservices architecture, asynchronous communication, secure transactions, and cloud-native deployment practices.

The platform integrates multiple technologies working together to provide a scalable and resilient e-commerce environment.

7.1 Frontend Technologies

The frontend layer was responsible for providing an interactive and responsive user interface for customers, vendors, and administrators.

React.js

React.js was used as the primary frontend framework because of its component-based architecture and high rendering performance.

React provides several advantages such as:

- Reusable UI components
- Virtual DOM rendering
- Faster page updates
- Better scalability
- Improved maintainability

The frontend application was divided into multiple reusable components including product cards, dashboards, shopping cart modules, and authentication pages.

Bootstrap 5

Bootstrap 5 was integrated to design responsive and mobile-friendly user interfaces.

Bootstrap simplified frontend styling through:

- Responsive grid systems
- Prebuilt UI components

- Flexible layouts
- Mobile responsiveness

This improved the overall user experience across multiple devices.

Axios

Axios was used for frontend-to-backend communication through REST APIs.

The Axios library helped in:

- Sending HTTP requests
- Attaching JWT tokens
- Managing API responses
- Handling exceptions centrally

It improved communication efficiency between the frontend and backend services.

React Router

React Router enabled client-side navigation within the application.

The routing system allowed:

- Seamless page navigation
- Protected routes
- Dynamic route handling
- Faster page transitions

This improved both user experience and frontend performance.



Figure 7: Frontend Technology Stack of InstaBuy

7.2 Backend Technologies

The backend layer was developed using Spring Boot microservices architecture. The backend handled business logic, distributed transactions, security, and communication between services.

Spring Boot

Spring Boot was selected because it simplifies enterprise application development through production-ready configurations and embedded server support.

Major benefits of Spring Boot include:

- Rapid API development
- Simplified dependency management
- Embedded Tomcat server
- Production-ready deployment
- Reduced boilerplate code

Each microservice in InstaBuy was implemented as an independent Spring Boot application.

Spring Cloud Gateway

Spring Cloud Gateway was implemented as the centralized API Gateway for all incoming client requests.

The Gateway handled:

- Request routing
- JWT validation
- Security filtering
- Load balancing
- Traffic management

This improved security and centralized communication management.

Netflix Eureka

Netflix Eureka was used for dynamic service discovery in the distributed environment.

It enabled:

- Automatic service registration
- Runtime service discovery
- Dynamic scaling
- Fault tolerance

This removed the need for hardcoded service URLs.

InstaBuy – Backend Architecture

Distributed E-Commerce Application

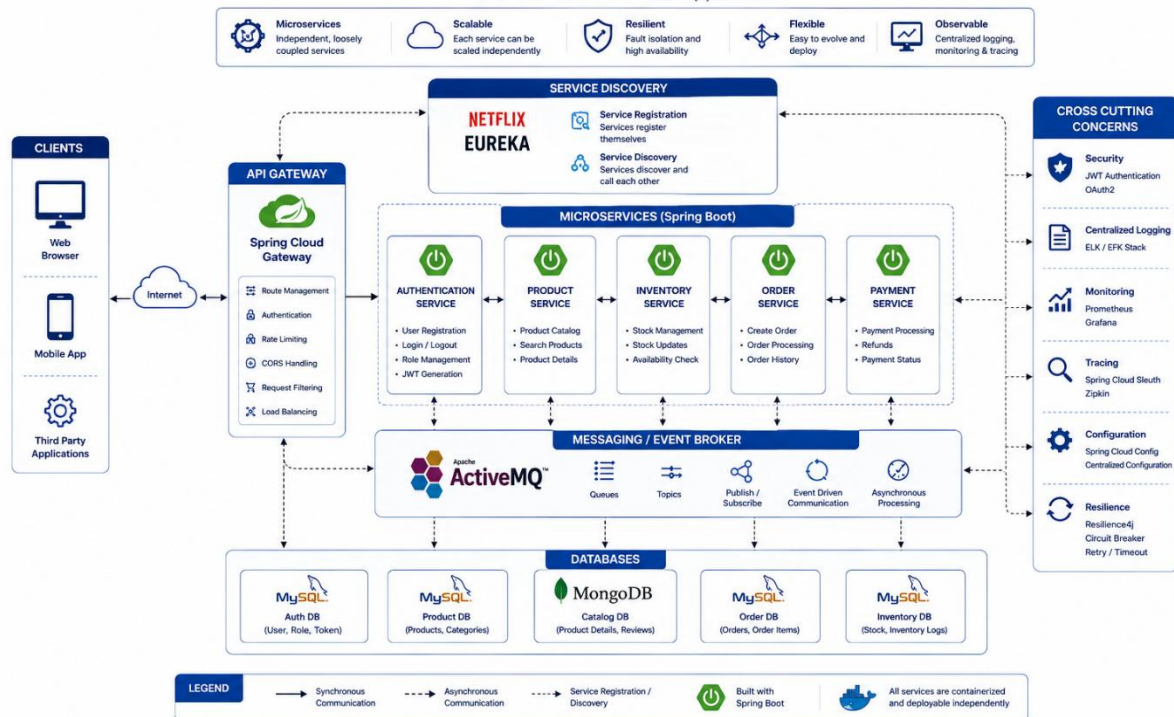


Figure 8: Backend Technology Stack and Service Flow

7.3 Database Technologies

InstaBuy uses both relational and NoSQL databases to optimize performance and flexibility.

The use of multiple databases allowed the system to efficiently handle both transactional and dynamic product-related data.

MySQL

MySQL was used for handling structured transactional data requiring strong consistency.

The database stores:

- User information
- Orders
- Payments
- Inventory records
- Role mappings

MySQL was selected because of:

- ACID compliance
- Reliable transaction handling
- Structured relationships
- Data consistency

This ensured secure and reliable processing of business transactions.

MongoDB

MongoDB was used for storing product-related data because product structures frequently change in e-commerce platforms.

MongoDB stores:

- Product details
- Product descriptions
- Product categories
- Vendor product listings

Advantages of MongoDB include:

- Flexible schema design
- High scalability
- Faster modifications
- Better handling of unstructured data

The combination of MySQL and MongoDB improved overall application performance and flexibility.

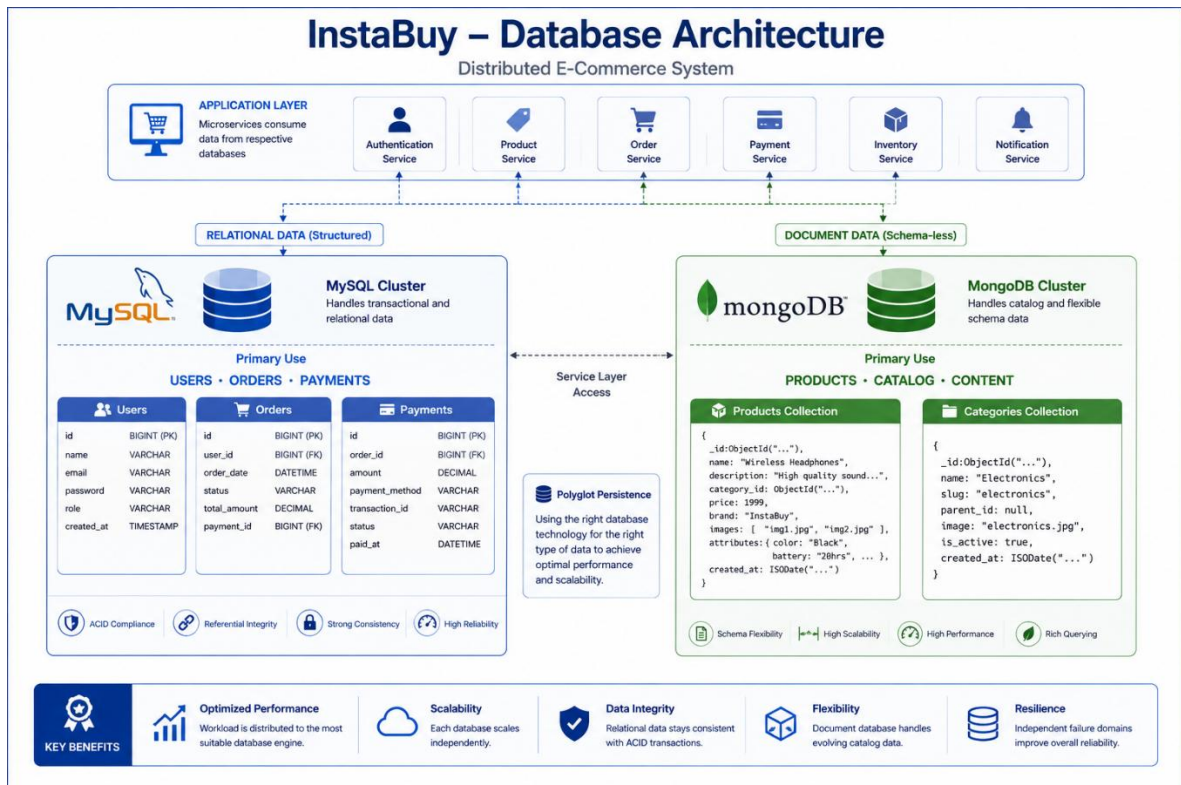


Figure 9: Hybrid Database Architecture Using MySQL and MongoDB

7.4 Messaging and Distributed Communication

One of the most important aspects of InstaBuy is asynchronous communication between microservices.

Apache ActiveMQ was implemented as the messaging broker to support event-driven communication.

Apache ActiveMQ

ActiveMQ enabled services to exchange events asynchronously through queues and topics.

The messaging system was mainly used for:

- Order processing
- Inventory reservation
-
- Payment confirmation
- Failure recovery

- Distributed transaction coordination

Instead of blocking services through synchronous communication, services communicate independently using events.

This architecture improved:

- Scalability
- Fault tolerance
- Reliability
- Service independence

Saga Pattern

The Saga choreography pattern was implemented for distributed transaction management.

The Saga approach helped:

- Maintain eventual consistency
- Avoid distributed locks
- Handle compensation transactions
- Manage rollback operations

This was especially important during order and payment workflows.

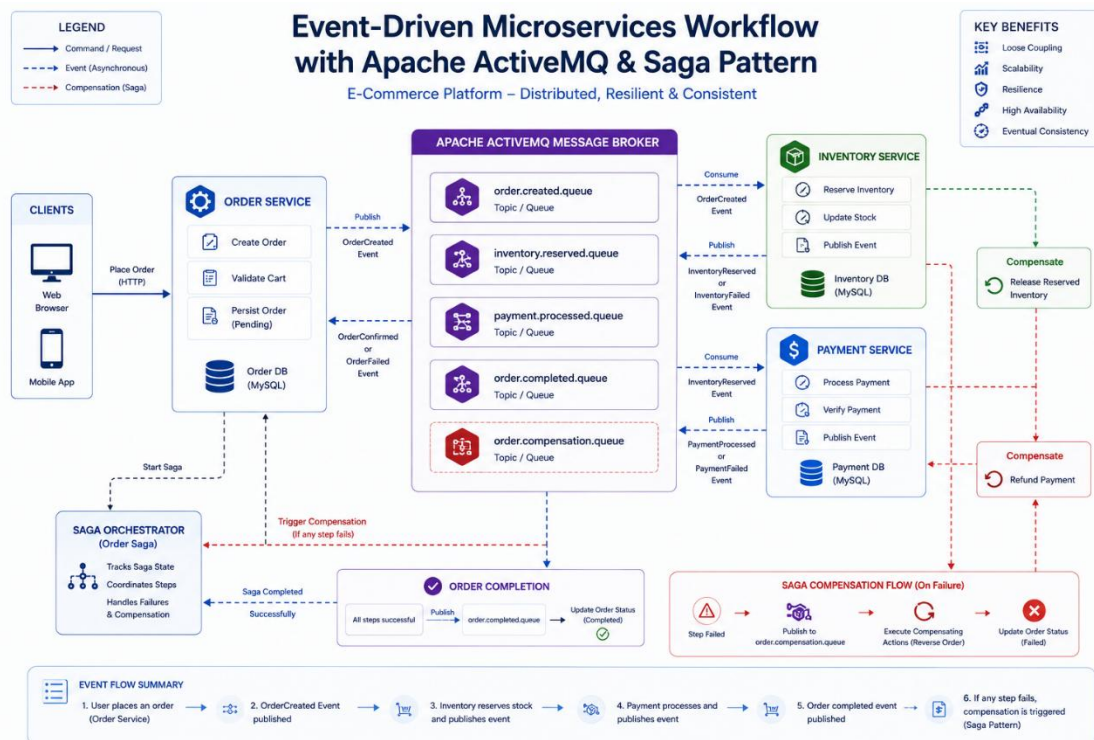


Figure 10: ActiveMQ Event-Driven Communication and Saga Workflow

7.5 Security Technologies

Security was implemented throughout the application to protect user data, transactions, and service communication.

JWT Authentication

JSON Web Tokens (JWT) were used for stateless authentication.

After successful login:

- The server generates a JWT token
- The token is attached to subsequent API requests
- The Gateway validates the token before forwarding requests

Advantages include:

- Stateless authentication
- Reduced server session management
- Improved scalability
- Secure request validation

BCrypt Password Encryption

User passwords were encrypted using BCrypt hashing before storing them in the database.

BCrypt improves security by:

- Preventing plaintext password storage
- Protecting against brute-force attacks
- Reducing password theft risks

Role-Based Access Control (RBAC)

RBAC was implemented to restrict access based on user roles.

The major roles include:

- Customer
- Vendor
- Administrator

This ensured that users could access only authorized functionalities.

7.6 Payment Integration

The InstaBuy platform integrates Stripe API for secure online payment processing.

Stripe was selected because it provides:

- Secure transactions
- PCI-DSS compliance
- Webhook support
- Real-time payment verification

The payment workflow includes:

1. User initiates checkout
2. Payment Service creates Stripe session
3. Stripe processes transaction
4. Webhook confirms payment result
5. Payment events are published to ActiveMQ

This architecture ensures reliable payment processing within distributed workflows.

8. FRONTEND IMPLEMENTATION

The frontend layer of InstaBuy was designed with the primary objective of providing a smooth, responsive, and user-friendly shopping experience for customers, vendors, and administrators. Since the platform follows a distributed microservices architecture, the frontend acts as a centralized interaction layer that communicates with multiple backend services through REST APIs routed via the API Gateway.

The frontend application was developed using React.js because of its component-based architecture, efficient rendering mechanism, and scalability. Bootstrap 5 was integrated to improve responsiveness and accelerate UI development. Axios was used for API communication, while React Router enabled seamless client-side navigation across different pages of the application.

The frontend was designed not only to improve user experience but also to support secure authentication, dynamic content loading, vendor product management, and real-time order interaction.

8.1 Frontend Architecture

The frontend follows a modular architecture where every major functionality is separated into reusable components and pages. This design improves maintainability, readability, and future scalability of the application.

Instead of building the application as a single large interface, the UI was divided into multiple independent components such as navigation bars, product cards, authentication forms, checkout pages, and dashboards. These reusable components reduced code duplication and simplified frontend management.

The frontend architecture mainly consists of:

- Reusable UI Components
- Page-Based Routing
- Centralized API Communication
- Authentication Handling
- Dynamic State Management

This structure allowed efficient interaction with backend microservices while maintaining a clean and scalable frontend codebase.

8.2 React.js Implementation

React.js was selected as the frontend framework due to its high performance and component-driven development model. React uses a Virtual DOM mechanism that updates only the modified portions of the webpage instead of reloading the entire interface. This significantly improves application speed and responsiveness.

The application interface was divided into independent React components. Each component was responsible for handling a specific UI functionality.

Some major frontend components include:

- Navigation Bar Component
- Product Listing Component
- Product Card Component
- Shopping Cart Component
- Checkout Component
- Login and Registration Forms
- Vendor Dashboard
- Order History Section
- Sales Analytics Dashboard

The use of reusable components simplified frontend development and improved overall consistency across the application.

8.3 Routing and Navigation

To improve user experience, InstaBuy implemented client-side routing using React Router. This enabled users to navigate between pages without refreshing the browser window, resulting in faster page transitions and smoother interaction.

The routing system handled:

- Product browsing pages
- Authentication pages
- Checkout pages
- Vendor management pages

- Order history pages
- Admin dashboards

Protected routes were implemented to ensure that unauthorized users could not access restricted pages such as vendor dashboards or payment pages.

For example, if a user attempted to access a protected route without authentication, the application automatically redirected them to the login page.

This routing mechanism improved both security and navigation efficiency.

8.4 State Management

Efficient state management was essential for handling dynamic user interactions such as cart updates, product searches, login sessions, and checkout operations.

React Hooks were used extensively for state handling within components.

The major hooks used include:

- `useState()`
- `useEffect()`
- `useContext()`

The frontend maintained states for:

- User authentication status
- JWT tokens
- Shopping cart items
- Product lists
- Order details
- Vendor inventory information

State updates were optimized carefully to reduce unnecessary rendering and improve application performance.

8.5 Axios-Based API Communication

The frontend communicates with backend microservices through REST APIs using Axios. Axios simplified HTTP communication by providing centralized request handling and better error management.

Every API request from the frontend passes through the API Gateway before reaching the appropriate backend service.

Axios was configured to automatically:

- Attach JWT tokens in request headers
- Handle authentication errors
- Process API responses
- Manage exceptions globally

This centralized communication model improved application security and reduced repetitive coding effort.

For example:

- Login requests were sent to the Authentication Service
- Product retrieval requests were routed to the Product Service
- Order creation requests were forwarded to the Order Service
- Payment requests interacted with the Payment Service

The use of Axios improved maintainability and simplified communication between frontend and backend layers.

8.6 Responsive User Interface Design

The frontend interface was designed using Bootstrap 5 to ensure responsiveness across multiple screen sizes and devices.

The system supports:

- Desktop devices
- Tablets
- Mobile phones

Bootstrap's responsive grid system allowed product cards, navigation menus, and forms to automatically adjust based on screen dimensions.

Special attention was given to:

- Product browsing experience
- Mobile-friendly checkout flow

- Responsive navigation menus
- Adaptive vendor dashboards

The responsive design significantly improved accessibility and user experience.

8.7 Security Features in Frontend

Frontend security was implemented carefully to protect user sessions and sensitive operations.

Several security mechanisms were integrated, including:

- JWT token-based authentication
- Protected frontend routes
- Automatic logout handling
- Secure API request headers
- Role-based page access

The frontend also restricted unauthorized access to vendor and administrator functionalities. Only users with valid roles were allowed to access protected sections of the application.

This ensured that:

- Customers could only access customer features
- Vendors could manage only their own products
- Administrators could access global system controls

These security practices improved overall platform protection and user trust.

9. BACKEND IMPLEMENTATION

The backend layer of InstaBuy was developed using Spring Boot microservices architecture. The backend serves as the core processing layer responsible for handling business logic, distributed communication, database operations, security, and transaction management.

Unlike monolithic systems where all functionalities exist inside a single application, InstaBuy separates major business capabilities into independent microservices. Each microservice operates independently and performs a dedicated task within the system.

This architecture improves:

- Scalability
- Fault isolation
- Independent deployment
- Maintainability
- System resilience

The backend services communicate using both synchronous REST APIs and asynchronous event-driven messaging through Apache ActiveMQ.

9.1 Spring Boot Microservices Architecture

Spring Boot was selected because it simplifies enterprise application development using production-ready configurations and embedded server support.

Each microservice was developed as an independent Spring Boot application with its own:

- Controllers
- Services
- Repositories
- Database connections
- Security configurations

This separation reduced dependency between services and enabled independent scaling of each module.

The core backend services include:

- Authentication Service
- Product Service
- Inventory Service
- Order Service
- Payment Service

Each service handles a specific business responsibility within the platform.

E-Commerce Platform – Microservices Architecture

Scalable • Resilient • Loosely Coupled • Independently Deployable

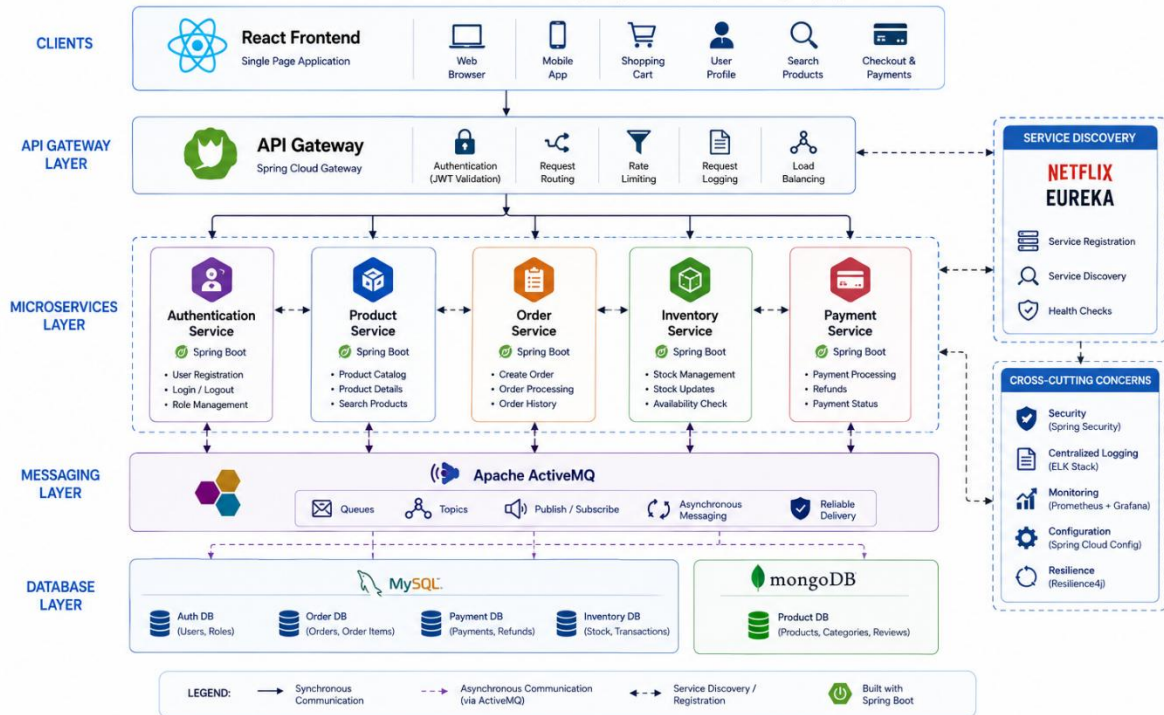


Figure 11: Spring Boot Microservices Architecture of InstaBuy